

TEST IT LIKE A USER

Five days. Your class is handed a working app with **six real usability problems built into it on purpose**, they find them by watching other people use it — and on the last day you fix one live and **make them test it again on somebody new**.

5

days, each ending in written evidence

6

findable problems, listed for you

0

accounts, logins or subscriptions

The app is included, and it is deliberately flawed.

A class cannot find usability problems in a polished app, and a class that finds nothing concludes that testing is a formality. So this one ships with six genuine ones, of the kind every first build has. **The teacher's guide lists all six**, so you can tell whether your class is finding real things or guessing.

The students never touch the chatbot. That is the better lesson.

Chatbots are 13 and over, and no child's name goes into a prompt. So students do the testing, the observing, the sorting and the deciding, and **you do the prompting, on the board, where they can see their own evidence becoming a change in the software**. Watching a person and arguing from what you saw is the thinking. Typing prompts is not.

Inside

Day-by-day teacher pages · **six student pages worth printing** · the three tester tasks written word for word · the six flaws with the sentence a tester is likely to say · the exact prompts for day 5 · what to do when the AI breaks the app in front of thirty children · and the working app, which has no name field anywhere.

BEFORE YOU START

The week at a glance

Day	The class does	You do
1 Watch	Give one person a task and watch without helping. Write down what they <i>did</i> , not what they said.	Model it badly first — helping, explaining, leading — then well.
2 Test	Three testers each, the same three tasks, silence. Findings on paper.	Enforce the one rule: if you had to explain it, that is a finding.
3 Sort	Group the findings. Decide which are real problems and which are one person's preference.	Push on "everybody said" when two people said it.
4 Choose	Pick three fixes out of everything found, and write the evidence for each one.	Refuse any fix with no observation behind it.
5 Fix and re-test	Watch the fix happen, then test it again on somebody who has not seen it.	Prompt on the board. Then make them test it again.

Day 5 is the one that makes this a unit rather than an activity.

Almost nobody re-tests. A class that fixes something and then watches it still confuse somebody has learned the actual lesson, which is that *a fix is a hypothesis*, not an ending.

How long it really takes

Days 1, 3 and 4 are a lesson each, and day 1 can be as short as twenty minutes. **Day 2 is as long as you can give it** — three testers each takes about twenty-five minutes once the class knows the rules. **Day 5 is thirty minutes**, split roughly in half between the fix and the re-test. The days do not have to be consecutive and the unit works better spread across two weeks.

What you need

One computer you can project. Any AI chat tool you already use, for day 5 only. Printed findings sheets. A handful of devices for day 2, or one device and a line of testers. And the app in this download, which opens by double-click and needs nothing installed.

Three rules, and the first one is the whole unit

1 • The tester is never helped and never named

The observer does not hint, does not point, does not say "it's the other button," and does not laugh. **The moment you help, the data is gone** — you have measured how good you are at explaining, which nobody needed to know.

And the findings sheet records **what the tester did, never who they were**. "Tester 2 pressed Submit and stopped" is a finding. Naming the child who struggled, in front of the class, is how this unit goes wrong, and the student pages say so too.

2 • The grown-up holds the keyboard

Every mainstream chatbot requires users to be 13 or over, and school guidance is consistent that a child's name, work or identity should not be typed into one. So **no student sits at the chatbot** at any point. Students specify, observe, argue and decide. That is the harder half and the half that transfers.

3 • Evidence beats opinion, including yours

A fix gets made because somebody was *watched* getting stuck, not because it would look nicer. Day 4 enforces this and it is the rule students will fight hardest. When a student cannot say who got stuck and where, the answer is not "no" — it is **"go and watch one more person."**

And one thing to settle before day 2

Testers should be people who have not seen the app.

Classmates who have been staring at it all week will sail through and find nothing. Another class, a sibling, a teaching assistant, the office staff, a parent at pickup. **If that is impossible, split your room in half** and keep one half away from the app until day 2.

The app you have been given, and why it is flawed

The download includes `makerspace-booker.html`. It books a shared room — pick a day, pick a slot, book it, and slots show as free or taken. **It works. It does not crash and it does not lose bookings.** It is also quietly bad to use, on purpose.

Why we did not ship you a good one

Usability testing on a polished app produces a class of thirty children writing "it was fine." Nothing is found, nothing is argued about, and the unit teaches that testing is a box to tick. **Every problem in this app is a real one**, of the kind that turns up in a genuine first build, and all six are findable by an eleven-year-old inside a two-minute task.

It is also the honest version of the lesson. **You cannot see these problems from the inside.** The person who built it knows which button books, so the button seems obvious. That gap is the entire reason user testing exists, and a flawed app teaches it in a way a perfect one cannot.

Do not fix it before the class tests it

The fixed version is what day 5 produces. If you repair the app over the weekend out of embarrassment, you have removed the unit. **The next page lists what is wrong with it** so that you know what your class should be finding — it is for you, not for them, and it should not go on the wall.

What the app deliberately does not have

There is no name field anywhere in it, and no record of who booked what. A booking system that records which named child was in which room, at which time, is a movement record about identified children, and we would rather not build one and then ask a class to improve it. Bookings are held against slots and a four-character code, and that is all.

If you would rather use your own app

Everything in this pack works on any app your class has already built, or on any app at all — the school website, a lunch ordering system, the library catalog. **The five days do not depend on the booker.** It is here so that the unit is complete on the day you buy it.

The six flaws, and what a tester will say

Read this, then put it away. **Knowing the answers is what lets you tell whether your class is finding real things or guessing**, and lets you steer a task toward a flaw nobody has hit yet.

#	What is wrong	What a tester tends to say
1	The big Submit button only saves a draft. A second, smaller button called Confirm booking is the one that actually books.	<i>"I thought I'd booked it."</i>
2	Nothing says it worked. After a real booking the screen looks almost identical — a four-character code appears in small gray text and that is the whole confirmation.	<i>"Did that work?"</i>
3	Every error message is the single word Error , whether the slot is taken, the day is missing or the code is wrong.	<i>"It won't let me and it won't say why."</i>
4	The steps only work in one order. Pick the day first, then the slot. Picking a slot and then changing the day silently clears the slot, and nothing says so.	<i>"It keeps clearing what I picked."</i>
5	Book the whole day is the biggest button on the screen and is almost never what anybody wants. Cancel a booking is the smallest thing on the booking screen.	<i>"I nearly booked the whole day by accident."</i>
6	The confirmation code is shown once, is wiped by the next thing you do, and is required on a different screen in order to cancel.	<i>"Nobody told me to write that down."</i>

If the class has found four of these by the end of day 2, the testing is working.

If they have found one, the problem is almost always that they are helping. Go back to day 1 for five minutes and model it badly again.

Steering without telling

Flaws 1, 2 and 4 come out of task one. Flaw 6 and flaw 3 come out of task two. Flaw 5 comes out of task three, or out of anybody who is in a hurry. **If a flaw is going unfound, add a tester and give them that task** rather than announcing it.

Watch somebody struggle, and say nothing

Twenty to thirty minutes. Ends with a room that knows the difference between watching a person and interviewing one, which is the skill the rest of the week stands on.

1 Do it badly, at the front (4 minutes)

Ask a volunteer to book Thursday afternoon on the projected app. Then **help them relentlessly**: point at the screen, say "not that one," explain what the button does, finish their sentence. Stop and ask the room what you learned about the app. **Nothing**. You learned that you can operate it, which was never in doubt.

2 Do it properly (4 minutes)

Second volunteer, same task, and now you say one sentence at the start and nothing after it: **"I can't help you, and nothing that goes wrong is your fault."** Sit on your hands. Let the silence be uncomfortable. The room will find it almost unbearable, and that reaction is worth naming out loud.

3 What to write down (5 minutes)

Model the difference on the board. **"They clicked Submit, waited about three seconds, then clicked it again"** is an observation. "They found it confusing" is not — it is your conclusion about them. The student sheet has one column for each, on purpose, and the left column has to be filled in first.

4 Everybody watches one person (12 minutes)

Pairs. One tester, one observer, one task: **book any slot on Wednesday**. Swap. Then collect two or three observations aloud and bin any that are conclusions. **Do not discuss the app yet**. Today is about watching, not about the booker.

The sentence to end on.

"If you had to explain it, that is a finding." Write it up and leave it up all week. It converts the helping instinct, which you cannot switch off, into the data you actually want.

If it derails

Somebody will tell their tester the answer within ten seconds. Do not tell them off — **ask the room what that cost**. The pair has to start again with a new task, and the point lands harder from the restart than from a reprimand.

Watch one person. Help nobody.

Your job is to write down **what the person did**, like a camera. Not what you think they felt, and not their name. Fill the left column first. Leave the right column blank if you are not sure.

Say this, then stop talking

"I can't help you, and nothing that goes wrong is your fault." After that: no pointing, no hinting, no faces.

The task I gave them:

What they did – like a camera

What I think it means

The hardest question, and the one worth answering

Where did they **pause**? Not where they failed — where they stopped and looked at the screen for a second.

Did I help?

☐ No, not once ☐ Yes, a bit ☐ Yes, a lot

If you helped, write what you said. **That sentence is a finding** — it is the thing the app should have said instead of you.

Run real tests on real strangers

One lesson, twenty-five minutes of it testing. Ends with a stack of findings sheets that are evidence rather than opinion. This is the day the unit lives or dies.

1 Set the testers up (3 minutes)

Each student needs **three testers who have not seen the app**. Another class, older or younger, is ideal and a younger class is better than an older one. Siblings, adults in the building and the office staff all count. If you must use classmates, use the half of your room you kept away from it.

2 Hand out the three tasks, word for word (2 minutes)

They are on the next student page and they are written out exactly because **a vague task produces a vague test**. Students read the task aloud, hand over the device, and stop talking. They do not demonstrate first.

3 Test (20 minutes)

Three testers, three tasks each, one findings sheet per tester. Circulate and listen for helping. **Say nothing about the app itself all lesson**, however much you want to, and especially do not confirm whether something they found is "right."

4 One sentence each, out loud (5 minutes)

Go round the room. **One observation each, in the form "my tester did X at Y."** Do not sort, do not agree, do not fix. Just hear them. Tomorrow is for arguing.

The forcing question, for "it was fine."

"It was fine" is not a finding, and you will get a lot of it. The question that breaks it every time is: **"What did they do, exactly, at the moment they paused?"** It is on the student sheet too, because it works better when they ask it of themselves.

If it derails

A tester who gets frustrated and gives up is **your best data**, and the room needs to hear you say that. Thank them properly, write down exactly where they stopped, and make sure the child who tested them knows they did the job right rather than badly.

The three tasks. Read them out exactly.

Read the task aloud, hand over the device, and **stop talking**. Do not show them first. Do not explain what the buttons do. One sheet per tester.

Task 1

"Book the makerspace for Thursday, first slot after lunch."

Task 2

"You booked Tuesday by mistake. Cancel it."

Task 3

"Find the earliest free slot tomorrow and take it."

Before you start, set the app up for task 2

Task 2 only works if there is a Tuesday booking. Book one yourself before the tester sits down, and **keep the code** — you may need it, and finding out why is part of the point.

The two rules, again

You may not help. Not with your hands, not with your face.

You may not write their name. Call them Tester 1, Tester 2, Tester 3.

If they ask you a question, write the question down.

Then say "I can't help with that, do whatever you think." **The question itself is the finding** — it is the thing the app should have answered without being asked.

Findings sheet – one per tester

Tester: ☐ 1 ☐ 2 ☐ 3 No names. Roughly how old: _____ Had they seen the app before? ☐ No ☐ Yes

Task	Where they got stuck, and what they did	Finished it?
1		Y / N / with help
2		Y / N / with help
3		Y / N / with help

Anything they said out loud, in their words

Anything they asked me

Every question is a finding. Write it exactly.

And the question that beats "it was fine"

What did they do, **exactly**, at the moment they paused?

Sort the pile, and argue about it

One lesson. Ends with every finding in one of three groups and a room that can tell a problem from a preference, which is a distinction most adults cannot make either.

1 Everything on the wall (8 minutes)

One finding per sticky note or per strip of paper, up on the board in no order. **Duplicates stay up** — the duplicates are the whole point, because a finding three testers hit is a different animal from one that one tester hit.

2 Group them, by what went wrong (12 minutes)

Not by task, by cause. Students come up and move things. Expect arguments about whether two notes are the same thing and **let them run**; working out that "I didn't know it saved" and "I pressed it twice" are the same underlying problem is the hardest thinking in the week.

3 Three buckets (10 minutes)

Label the groups: **Real problem** (more than one tester, or one tester who failed the task), **Just a preference** (somebody would have liked it differently), **Not sure yet** (needs one more tester). The third bucket has to exist or everything gets forced into the first.

4 Count, out loud (5 minutes)

For each real problem: **how many testers hit it, out of how many?** Write the number on the note. From here on, a finding without a number next to it does not get argued for.

Push hard on "everybody said."

Nobody ever says "two of my three testers." They say "everybody said." **Make them count**, every single time, and make them say the fraction aloud. It is the most useful habit in the unit and it survives long after they have forgotten the app.

The one to watch for

A student will argue that something is a real problem because *they* find it annoying, having used the app all week. **That is a preference, and from the worst possible source** — they are the only people in the building who already know how it works. Ask who they watched.

Real problem, or just a preference?

Take your findings one at a time. A finding is a **real problem** if more than one tester hit it, or if one tester could not finish the task because of it. Everything else goes in one of the other two columns, and that is fine.

The finding, in a few words	How many testers	Which column
		R / P / ?
		R / P / ?
		R / P / ?
		R / P / ?
		R / P / ?
		R / P / ?

R = real problem · P = just a preference · ? = not sure, needs one more tester

The argument I had, and who won

Write down one finding somebody disagreed with you about, and what settled it.

Say the fraction out loud

Not "everybody said." "Two of my three testers." If you cannot say the fraction, the finding is not ready to argue for yet.

Pick three, and justify every one

One lesson. Ends with exactly three fixes, each with a named piece of evidence behind it, and a room that has had to let go of two things it wanted.

1 Why three and not everything (4 minutes)

Because that is the real constraint. There is always more wrong than there is time to fix, and **choosing is the job**. Say the number out loud and do not negotiate it. A class that is allowed six will fix six badly.

2 Nominations, with the fraction (10 minutes)

A student nominates a fix and must say: **what the fix is, who got stuck, and how many out of how many**. Missing any of the three and the nomination does not go on the board. This feels harsh for about four minutes and then the room adapts.

3 Argue and cut (12 minutes)

Push one question: **"If we only fix one, which one, and why that one?"** Expect them to pick the thing that annoys them rather than the thing that stopped a tester finishing. Keep pointing back at the task-completion column on the sheets.

4 Write the three as instructions (8 minutes)

Not "make it clearer." **"When a booking is made, show a green message that says Booked, with the day and the time in it."** A fix you cannot hand to somebody else is not a fix yet, and tomorrow you have to hand one to a chatbot.

The two they let go of are worth a minute.

Write the rejected two on the board under the heading **Next time**. Nothing about this unit promises there will be a next time — but a class that sees good ideas parked rather than binned argues better the following week.

If they pick a fix nobody was stuck on

Do not veto it. **Ask who got stuck on it and wait**. The silence does the work. If a student can produce a tester and a moment, it stays in — even if you think it is wrong, because the rule has to apply to you too.

Three fixes, and the evidence for each

You get three. Not four. For each one, write **what to change** as an instruction somebody else could follow, then **who got stuck** and **how many out of how many**. A fix with an empty evidence box does not count.

Fix 1

Change this:

Because I watched:

How many testers hit it: _____ out of _____

Fix 2

Change this:

Because I watched:

How many testers hit it: _____ out of _____

Fix 3

Change this:

Because I watched:

How many testers hit it: _____ out of _____

Is a fix written well enough?

"Make it clearer" is not a fix. "When a booking is made, show a green message saying Booked, with the day and time in it" is. Could a stranger do what you wrote without asking you one question?

The fix I had to give up, and why it lost:

Fix one live – then test it again

Thirty minutes, split in half. The first half is the fix. **The second half is the part almost nobody does**, and it is the reason this unit exists.

1 Read the fix out, from their sheet (2 minutes)

A student reads the chosen fix aloud as written, with the evidence after it. **You type what they wrote**, not an improved version of it. If the instruction is too vague for the AI to act on, that is a finding about the instruction, and the room should see you hit it.

2 Prompt on the board (10 minutes)

Prompts are written out in full on the next page. Project the chat. **Let it go wrong where they can see** — the recovery is more useful than a clean first try, and the class watching you say "no, that is not what we asked for" is the whole lesson about who is in charge.

3 Save the fixed file, before you test it (1 minute)

Press **Save this booker as a file** and keep it somewhere you will find it. Save the original too, under a different name. **You want both**, because the comparison is the evidence.

4 Re-test on somebody new (15 minutes)

A tester who has never seen either version. Same task that exposed the problem. Same silence. The student sheet for this is the last student page in the pack, and it asks one question that matters: **did the new person get stuck in the same place?**

If the fix made it worse, you have had the best possible lesson.

Say that out loud and mean it. A class that watches its own confident, well-evidenced fix confuse a new person has learned something that cannot be taught any other way: **a fix is a hypothesis**. Do not rescue the moment by fixing it again.

If you have another twenty minutes

Do the second fix and re-test that one too. The second round is visibly faster and much calmer, and the class notices that themselves, which is worth more than you pointing it out.

The prompts, written out in full

You are at the keyboard. Open your AI chat tool, **attach or paste the whole** `makerspace-booker.html` file, and use these in order. The parts in yellow are the only parts you change.

PROMPT 1 · THE FIX

This is a single-file HTML app that books slots in a room.
Make exactly ONE change and nothing else:

`<paste the fix, in the class's own words>`

Rules:

- It must stay one HTML file that opens by double-click and works offline.
- Do not add any field that asks for a person's name.
- Do not change anything I did not ask you to change.
- Give me back the whole file, not a snippet.

PROMPT 2 · WHEN IT DID SOMETHING ELSE AS WELL

You changed more than I asked. Start again from the file I gave you
and make only this change: `<the one fix>`
Leave every other button, color and word exactly as it was.

PROMPT 3 · WHEN THE NEW VERSION IS BROKEN

That version does not work. When I `<what you did>`,
`<what happened instead>`.
Go back to the version before your last change and fix only that.

Worked example, for the fix most classes choose

If the class picks the missing confirmation, the middle of prompt 1 reads: **"When a booking is made, show a green message under the buttons that says Booked, followed by the day and the time. Keep showing it until the next thing the user does."** That is what a well-written fix looks like — a stranger could do it, and you can tell whether it worked by looking.

The sentence to say while it is generating.

"If this comes back different from what we asked, **we are right and it is wrong.**" Children assume the computer knows better. Day 5 is the only chance all week to say otherwise while they watch.

Re-test the fix on somebody new

Find a tester who has **never seen this app**, in either version. Give them the task that broke before. Say the sentence, then stop talking.

What we changed

The task I gave them:

What they did – like a camera

Stuck in the
same place?

Yes / No

Yes / No

Yes / No

Did the fix work?

☐ Yes ☐ No ☐ It fixed that, and broke something else

How I know:

And the one worth being honest about

Did anything get worse? A fix that fixes one thing and confuses somebody somewhere else is completely normal and is *not* a failure. Write it down anyway — especially if it was your fix.

When the AI breaks it in front of thirty children

It will happen, probably on the second prompt, and it is the most valuable five minutes in the unit if you handle it in front of them instead of apologizing for it.

It returned a snippet instead of the file

Most common by far. Ask again: **"Give me the complete file, starting with the doctype line and ending with the closing html tag."** Do not try to paste a fragment into the file yourself in front of a class.

The new version opens to a blank page

Something is unfinished in the code. **Close it, reopen the file you saved before the change**, and use prompt 3. This is the reason step 3 of day 5 says to save first, and the class will understand why instantly.

It fixed it and redesigned everything else as well

Very common and worth naming. Use prompt 2. Then ask the room the real question: **"Did anybody test the things it changed?"** No. So those changes are guesses, and guesses are exactly what the week has been arguing against.

It argues that your fix is a bad idea

Sometimes it is right. Read what it said to the class, let them decide, and **do whatever they decide**, including overruling it. The decision belongs to the people who watched the testers.

The one thing not to do.

Do not quietly fix it yourself at lunchtime and present a working version. **The class's relationship with the software depends on believing their evidence changed it**, and that is worth more than a tidy demonstration.

Opening it, resetting it, keeping it

Open `makerspace-booker.html` by double-clicking it. **You never edit code and you never open it in a text editor.** It works with the wifi off and nothing is transmitted anywhere.

What it does

Five days, six slots a day. Pick a day, pick a slot, and one of the two buttons books it. Booked slots show as taken and cannot be booked twice. Canceling needs the four-character code. **Bookings survive closing the browser**, which matters because the class needs the same state on day 2 that it had on day 1.

Clearing it between testers

Clear all bookings is at the bottom and asks before it does it. Use it between classes. Leave a few slots booked before a tester sits down, though — an empty grid makes task 2 impossible and makes task 3 too easy.

Saving your version

Save this booker as a file hands you back a single HTML file with your bookings inside it. Save the original before day 5 and the fixed one after, under different names. The two files side by side are the clearest evidence of the week that you will get.

What it will never ask for

There is no name field anywhere in the app and nowhere to type one. Bookings are held against a day, a slot and a code. If a class decides on day 4 that it should record who booked, that is a genuinely good instinct, and the page headed *What this unit refuses to record* is the answer to it.

Tested before it shipped, and the flaws were tested too.

The app was driven start to finish in a browser: booking, the draft button that does not book, every error message, canceling with a wrong code and then a right one, booking a whole day, reloading, and saving to a file and reopening it in a clean browser with the bookings intact. **All six flaws were confirmed present and reproducible**, which is an unusual thing to test for, and the console was checked for errors because a crash is not a usability finding.

The six ways this unit goes wrong

1 Students help the tester

The hardest rule in the unit and the one that destroys the data. Day 1 exists entirely to make the helping instinct visible before it ruins day 2. **Expect to restart at least one pair**, and do it without irritation.

2 "It was fine" findings

Not a finding. One question fixes it: **what did they do, exactly, at the moment they paused?** It is printed on the student sheet so that they can ask it of themselves while you are across the room.

3 The class finds nothing

Almost always helping, occasionally testers who already know the app. **You have the list of all six**, so add a tester and aim them at a task that walks into one — without announcing what they are supposed to find.

4 The class finds everything and fixes nothing

Thirty findings and no decision. Day 4 caps it at three with evidence required, which is the real-world constraint rather than a classroom one. **Do not negotiate the number.**

5 The fix makes it worse

This is a success. It is the only way a class learns that a fix is a hypothesis. Resist every instinct to repair the lesson, and put it on the wall.

6 Testers are classmates who already know it

They will finish every task in forty seconds and find nothing, and the class will conclude the app is fine. Another class, a sibling, an adult. **Failing that, split your room in half on day 1** and keep one half away from the app.

And one that is not a failure mode, although it feels like one.

A tester who gets frustrated and gives up is the best data in the room. Thank them, write down where they stopped, and make sure the child who tested them knows they did the job right.

Assessment, honestly

There is no quiz here and there should not be. What a class produces in this unit is observable, and these are the four things actually worth looking at.

What you are looking at	Not there yet	Got it
Observing Day 1 and day 2 sheets	"They found it confusing." A conclusion with no moment attached.	"They clicked Submit, waited, clicked it again, then looked at me."
Not helping Watched, on day 2	Hints, pointing, finishing the tester's sentence.	Silence, and the question written down instead of answered.
Arguing from evidence Day 3 and day 4	"Everybody said." "It's annoying."	"Two of my three, and one of them gave up on task 2."
Specifying a change Day 4 sheet	"Make it clearer."	An instruction a stranger could follow without asking anything.

The single best piece of evidence in the week

The day 5 re-test sheet, where a student has to write down whether their own fix worked. **A student who writes "no, and it broke something else" has understood more than one who writes "yes."** Mark it that way and say so before they fill it in.

What not to grade

How many findings a student produced. A quiet observer with three precise observations has done better work than someone with eleven vague ones, and **counting rewards exactly the wrong behavior** on the one day of the week where precision matters most.

If you need a grade in a book

Use the day 4 sheet. Three fixes, each with a change written as an instruction and an evidence line with a fraction in it. **Six boxes, each either filled properly or not**, which is defensible to a parent and takes about ninety seconds a child.

What this unit refuses to record, and why

Two decisions were made before anything was built, and both of them are about where information lives rather than about whether a name appears.

Build freely – none of this is about a child

The booking grid, the slots, the room, the three tasks, the findings sheets with their numbered testers, the three chosen fixes. It is all information about an app. **Prompt with it, print it, put it on the wall.**

Your own device – the saved files

The booker you save and the fixed version your class produces live on the computer you saved them to and are transmitted nowhere. **Treat them the way you treat any classroom file**, which for these is not very carefully, because there is nothing in them.

Stop – any of it attached to a named child

"Tester 2 pressed Submit twice" is a finding. **Writing which child that was, and reading it out, is a different thing entirely** — it is a record of a named child struggling, produced in public, by their classmates. The student pages say no names twice, and it is worth saying a third time on day 2.

And why the app has no name field

Build it so that the wrong thing is impossible, rather than so that you are careful.

A booking app that records which named child was in which room at which time is a movement record about identified children. We could have shipped one and written a warning next to it. **Instead there is nowhere to type a name**, which is a rule nobody has to remember on a busy Tuesday.

If your class decides it should record who booked

That is a good instinct and the right answer is not "no." Ask the two questions that decide it: **where would that list live, and who could read it?** A class that works out for itself that a list of who was where is a different kind of thing from a list of free slots has had a better lesson than any rule you could have given them.

What else is out there, and what is different here

Being straight about this is the fastest way to keep a review. Here is the comparison, in both directions, including the part that is not flattering.

What already exists

Design-thinking units, which usually run empathize, define, ideate, prototype, test — and in most classrooms the testing step is a single afternoon at the end. Genius hour and project-based units, which end at a showcase. Plenty of material on gathering peer feedback. **If what you want is a full design-thinking unit, those are broader than this.**

What is different about this one

It is the testing step, stretched to five days. That step is the one that gets compressed to nothing everywhere else, and it is where the actual discipline lives.

Something is actually wrong with the app. Peer feedback on each other's good work produces politeness. A flawed app produces findings.

It ends with a re-test. Almost nothing does, and that is the difference between a class that has done testing and a class that has understood it.

What this pack is not

Not a design-thinking unit — there is no ideation stage and no prototyping. Not a course in programming. Not a professional usability method, which involves sample sizes and protocols this deliberately does not. And **not a claim that the booker is good software**, which would be a strange claim to make about an app we broke on purpose.

On building on somebody else's idea

Usability testing, booking grids and sticky-note sorting belong to nobody and you are welcome to build on all of them. The words, the design and the code in these files are ours. That distinction is worth teaching the moment a class starts changing software with AI: **how a thing works is free to borrow; the particular way somebody said it and drew it is not.**

License, credit, and one favor

What you may do

Run this unit with your own classes for as long as you teach. Print the student pages as many times as you need. Change the booker however you like and save as many versions as you want. Put the file on the classroom computers or a flash drive. Your one purchase covers you and your classes.

What needs another license

Sharing it with other teachers, putting it on a shared drive or a server, or handing it round a department or a building. Additional licenses are half price on the listing, and a school can buy them on a single purchase order. Please do not post the files publicly or sell anything made from them.

What was tested before this shipped

The booker was driven start to finish in a browser: a booking made with the draft button and then with the real one, every error message, a wrong cancellation code and then a right one, a whole day booked, a reload with the bookings intact, and the file saved and reopened in a clean browser. **All six flaws were confirmed present and reproducible**, and the console was checked for errors, because an app that crashes teaches nothing about usability.

If this earned its keep, would you leave a review?

TPT gives you credit toward your next purchase for it, and it is the only thing that decides whether another teacher ever sees this. If something did not work in your room, tell us that instead and we will fix it.

Built and audited by two certified teachers

Written by classroom teachers, checked by a second teacher before it shipped, and built on a rule we keep everywhere: the grown-up holds the keyboard, and no child's name goes into a prompt.